Extending an Agentic Coding Harness with a Conversational Surface, a Smart Memory, and Literature Capture

Authors:** Zeng Pu · zengpu987@gmail.com · Independent Researcher

Abstract

Agentic coding harnesses present a terminal-like, tool-first surface in which every user utterance is a task for an autonomous loop. That surface fits three needs poorly: casual conversation with a model, durable memory of material seen earlier, and capture of literature encountered while working. We describe an extension suite for DeepSeek Harness (DSH) that addresses them without forking it. The suite has three bundles: *chat mode*, a conversation panel with attachments and Markdown plus LaTeX typesetting; *Superman*, a smart-folder memory that classifies uploads, stores conversation and task records, and injects a dynamic digest into every agent prompt; and an *arXiv bookmarking* addition that archives a paper's PDF into that memory and degrades to a metadata record when the download is blocked. All three are contributed through the harness's slot system and an authenticated same-origin Fetch channel, so no shipped interface is replaced and no listening socket is added. The suite is 7 187 lines of JavaScript across six files. We verify it with 284 offline checks over five suites and a 14-file PDF corpus from which the hand-written reader extracts 199 942 characters (67 235 CJK) over 138 pages with zero noise, in 34.7 ms for a 19-page document and about 72 ms for two 54–56-page Chinese documents. Seven of seven encoding probes and sixteen of sixteen math expressions are handled correctly.

Keywords:** agentic coding harness, plugin architecture, slot-based UI extension, PDF text extraction, character encoding detection, MathML, lexical retrieval, long-term memory

1 Introduction

Interactive coding agents are usually delivered as a harness: a long-running process that owns a model connection, a tool registry, a session log, and a user interface. DeepSeek Harness (DSH) [1] follows this pattern, with an interface organised around sessions and turns whose primary affordances are the prompt box, the tool-call transcript, and the diff view. The design is effective for its target task: delegating bounded engineering work to an autonomous loop.

The same design leaves three gaps. First, **no plain conversational surface**: a user who wants to ask a conceptual question without granting the agent tools must accept the agent loop or leave the application, and the session model is no substitute, because sessions carry tool permissions and long system prompts irrelevant to a single explanatory exchange. Second, **no durable memory of material**: files passing through a session — a scanned contract, a screenshot, a paper — are attachments owned by that session, so a later session cannot discover that the material exists. Third, **no literature capture path**: DSH ships an arXiv search section, but a search result can be read, not kept.

Each gap could be closed by patching the harness, which is the wrong move for a research prototype: a fork diverges from upstream, must be rebased, and cannot be installed by a user who already has the application. We therefore ask whether all three capabilities can be added *from outside* — as extension bundles that occupy documented extension points, never replace shipped entries, and never modify harness source.

Our contributions are:

- **A slot-based extension pattern** adding three sidebar entries, two full-panel views, one settings section, and one overlay stack while leaving every shipped entry in place (Section 3).
- **An authenticated transport pattern** over exact Fetch routes on the existing `/api` channel, with a Server-Sent Events (SSE) [7] framing whose cancellation propagates into the provider call (Section 3).
- **A self-contained PDF text extractor**, written because the browser half may not import other Harness client packages: object-table scan, `DecompressionStream` inflation, `/ToUnicode` CMap parsing, content-stream replay, object streams, and a CJK fallback for fonts misdeclared `WinAnsiEncoding` (Section 4).
- **An encoding detector** with an explicit per-character plausibility score, and **a LaTeX-to-MathML translator** over a defined subset, chosen over bundling an existing

renderer (Section 4).

• **A memory subsystem** with an index-plus-files store, atomic write-then-rename persistence, a five-stage classification cascade, lexical recall, and prompt injection through a *function-valued* system-prompt section (Section 5).

• **An evaluation** over a 14-document PDF corpus, seven encoding probes, and a sixteen-expression math battery, recording the three bug classes the corpus exposed (Section 7).

2 Background and Related Work

2.1 The harness and its plugin framework

DSH is a Node.js application whose interface is a browser application served from `http://127.0.0.1:19387`. Extensions are Cordis [2] plugins: modules exporting an `apply(ctx, config)` function, where the framework supplies a context for declaring dependencies, registering services, subscribing to events, and contributing interface elements. A DSH *bundle* packages a host half (Node) and, optionally, a client half (an ES module executed in the page) with a manifest and a configuration patch inserting the host row into the profile.

Two properties matter. Interface contribution is **additive**: a plugin injects into a named *slot* and registers a component under an id of its choosing, so shipped slots keep their occupants. And activation is split: a **new** bundle becomes live through the loader without a restart, while an **edited** host half requires one.

2.2 PDF text extraction

Extracting text from PDF [3], [4] requires locating indirect objects, decoding filters such as `FlateDecode`, mapping character codes to Unicode through the font's `/ToUnicode` CMap, and replaying content-stream text operators (`BT`, `Tf`, `Td`, `TJ`, `Tj`, `ET`). Real documents add indirect `/Length` values, dictionaries inherited through `/Resources`, and page and font dictionaries existing only inside `/ObjStm` object streams. Mature libraries exist, but the client half of a DSH bundle is deliberately isolated (Section 3.4), so the suite implements the reader itself.

2.3 Math typesetting

Model output contains LaTeX [8]. Two browser strategies exist: convert to MathML [5] and let the engine lay it out with a system math font, or bundle a renderer such as KaTeX [9]. The first adds no third-party code; the second has broader coverage. The suite chooses MathML and a defined subset (Section 4.3).

2.4 Retrieval and recall

Recall over a small personal corpus can be lexical or dense. Lexical scoring in the tradition of BM25 [10] is deterministic, cheap, and model-free; dense retrieval generalises better but adds a model dependency. The suite's recall is lexical with explicit weights and a recency term (Section 5.3). Prior harness memory approaches range from session-log summarisation to external vector stores; the design point here is a *filesystem-visible* memory — an index plus original bytes under a fixed directory — which the agent reads with its ordinary file tools.

2.5 Literature metadata

The arXiv API [11] returns an Atom feed with title, authors, categories, abstract, and comment/journal-ref fields. Bookmarking must also obtain the PDF; whether the page can fetch it directly is a question about cross-origin headers, which we answer by measurement (Section 6).

3 System Architecture

Figure 1 shows the topology: three browser panels, the authenticated Fetch routes they call, and the host halves behind those routes. The suite is installed into a profile by the harness's bundle installer; no DSH source file is modified.

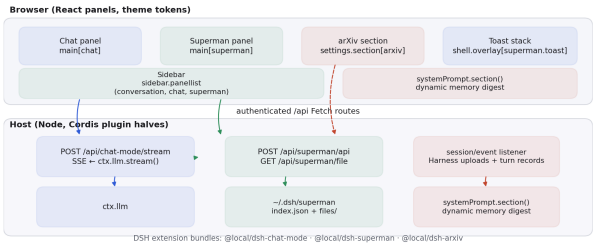


Figure 1. Extension topology: browser panels, authenticated Host routes, and the Cordis plugin halves.

3.1 Slots, and why shipped interface is never replaced

User-visible contribution uses `ctx.slots.inject(key, callback)` to declare interest in a named region and `ctx.slots.register(options, component)` to contribute a component with an id relative to it. Table 1 lists the occupancy. The chat bundle contributes two sidebar entries: `conversation` (order –20), which selects the harness's own shipped conversation panel rather than copying it, and `chat` (–10), the new panel; Superman contributes `superman` (–5). Negative orders put the mode switch before the shipped `plugins` and scheduled-task entries without displacing them. The suite also registers two `main`-slot keys (`chat`, `superman`), one `settings.section` (id `arxiv`, order 20), and one overlay (id `superman.toast`).

Table 1. Slot occupancy of the extension suite.

slot	entry id	order	purpose
sidebar.panellist	conversation	-20	Harness mode entry
sidebar.panellist	chat	-10	Chat mode entry
sidebar.panellist	superman	-5	memory entry
main	chat	—	conversation panel
main	superman	—	memory panel
settings.section	arxiv	20	arXiv search section
shell.overlay	superman.toast	—	remembered-file notices

Additivity makes the suite installable into an existing profile: removing a bundle restores the shipped interface untouched, and keeping it costs no shipped affordance. The cost is expressiveness: a capability needing a new first-class harness concept could not be delivered this way.

3.2 Transport: exact routes on the existing authenticated channel

The browser cannot call a provider directly because the credential is host-only, so the suite uses the harness's `/api` Fetch channel: `ctx.connection.fetch.register({path, methods, requestBody, fetch})` registers a handler for an exact path, and the caller inherits the host/origin trust fence and the session credential. No new listening socket is opened, so there is no extra port to firewall and no extra authentication code to get wrong.

Four routes are registered: `POST /api/chat-mode/stream`; `POST /api/superman/api` for `list`, `store`, `get`, `delete`, `note`, and `recall`; `GET /api/superman/file?id=`; and `GET /arxiv/search`, a same-origin proxy for the upstream Atom feed, because `export.arxiv.org` returns no `Access-Control-Allow-Origin`. Both halves use one JSON envelope, `{ok: true, value: …}` or `{ok: false, error:`

{code, message}}, with status derived from the code (bad-request 400, not-found 404, too-large 413, otherwise 500), so the panel can word every failure in the user's language.

3.3 Streaming and cancellation

Chat answers stream: the host half builds a ReadableStream and writes one SSE frame per model chunk.

```
data: {"type":"text","text":"…"}\n\n
```

Frame types are text (from text-delta), reasoning (from reasoning-delta), usage, finish, and error. A terminal finish whose reason kind is error or aborted becomes an error frame carrying the provider's failure code and message, so an aborted generation and a failed one stay distinguishable instead of both appearing as a silent stop. The response sets content-type: text/event-stream; charset=utf-8 and cache-control: no-store. The page reads the body with fetch and getReader(), reassembling frames on the double-newline separator so a chunk boundary inside a frame is handled. Cancellation propagates: request.signal goes both to the frame writer, which stops enqueueing once aborted, and into ctx.llm.stream, so pressing stop aborts the provider call.

3.4 Client-half isolation and hot reload

A client half may import React [14] and nothing else from the harness's client package set. That is the largest driver of code volume, since the PDF reader, encoding detector, LaTeX translator, Markdown renderer, and memory panel all live inside the bundles. We treat it as a feature — a bundle depending on another client package breaks when that package's internals move — but state its cost in Section 8.

Client halves are plain ES modules served by the client-module graph, and the development watcher polls each bundle's modification time and size every 500 ms and pushes changes to the page, so interface work needs no restart. Host halves are ordinary Node modules loaded by the Cordis loader: a new bundle activates immediately, but editing a loaded host half requires a restart. That asymmetry shaped our workflow, since almost all iteration happened in client halves.

4 Chat Mode

4.1 Conversation surface

Chat mode is one full-page panel with a collapsible conversation column, a centered message stream, and a composer. It calls no tools: the host half validates {provider, model, reasoningEffort?, messages} and forwards to the provider, taking model and thinking-effort choices from the harness's model catalogue. Validation is total: the route rejects a missing provider (provider-missing), missing model (model-missing), missing or non-array message list (messages-missing), more than 200 messages (messages-too-many), text over 400 000 characters (text-too-long), a list with no usable content (messages-empty), an image whose media type is not PNG/JPEG/WebP/GIF or whose bytes exceed 8 MiB (image-rejected), a non-JSON body (body), and any method but POST (method). Accepted images become durable attachment references rather than base64, and assistant turns are stamped with the model source the adapter requires.

4.2 Attachments, decoding, and the PDF reader

The composer accepts attachments by picker, drag-and-drop, and paste, up to six per message. Mathematica notebooks (.nb) are parsed into input and prose cells with box markup flattened to the code it displays; packages and scripts are inlined unchanged; HTML is reduced to visible text; images become attachment references; other textual input goes through encoding detection; unsupported and binary types are reported as skipped with a reason.

Encoding detection is ordered and scored. A byte-order mark settles UTF-8, UTF-16LE, or UTF-16BE immediately. Otherwise strict UTF-8 is accepted when it succeeds with no NUL and plausibility at least zero; if more than one eighth of sample positions are NUL, UTF-16 without a mark is tried in the parity-implied endianness and accepted above 0.4; otherwise GBK, Big5, Shift-JIS, EUC-KR, and windows-1252 [6] are decoded strictly and scored, and the best is taken only above 0.1, below which the file is reported as binary rather than inlined as mojibake. The score is a weighted per-character average: replacement characters and C0/C1 controls other than tab/newline/carriage return cost 6, CJK ideographs earn 3, kana and Hangul 2, printable ASCII and general punctuation 1, and Latin-1 supplement through IPA extensions cost 1. Dividing by the character count makes candidates comparable, which is why a GBK notebook does not lose to a windows-1252 reading that is "valid" everywhere.

The PDF reader, the largest component, has four phases. **Scan:** a pattern finds n g obj headers in a one-byte-per-character view, capturing each object's dictionary text and raw stream bytes, and /Filter is read as one complete token (name, array, or reference) with filters applied in reverse order — FlateDecode through the platform DecompressionStream (as deflate, then deflate-raw), ASCII85Decode and ASCIIHexDecode in the bundle, and any other filter marking the stream unusable for text. **Decode:** streams containing beginbfchar or beginbfrange become code-to-text maps, including array-valued ranges, merged first-definition-wins, and objects of /Type /ObjStm are decoded with their contents recovered from the /N and /First header. **Replay:** per page, /Resources fonts are resolved through indirect references and page-tree inheritance, and content streams are replayed with literal and hex strings read, two-byte codes handled for Type0 fonts, and codes mapped through /ToUnicode. **Assemble:** lines that decoded into noise are dropped and surviving pages joined, a result shorter than two characters being reported no-text and one below a 75% text ratio unreadable, so a scanned document yields an honest refusal rather than replacement characters; a fallback covers an unusable page tree by replaying every decoded stream containing both BT and a text-showing operator through the merged CMap.

Three bug classes found in development were each detected differently. **Trailing padding after the compressed data:** a stream may declare a /Length extending beyond the end of the zlib stream, making DecompressionStream raise; a naive reader discards the whole stream, ours treats the exception as end-of-input and returns the bytes already produced — detected by comparing character counts against a reference extraction, where our reader was systematically short. **Indirect /Length and /Filter over-reading:** /Length 12 0 R cannot be resolved at scan time, and a value reader consuming the rest of the dictionary swallows the keys after it, so the reader distinguishes an indirect length, locating data by endstream, from a direct one and reads one complete token per key — detected by a hand-built sample whose /Filter array is

followed by another key. **Inherited `/Resources` and dictionaries inside object streams:** modern files often omit `/Resources` on the page dictionary, so a reader looking only at top-level objects finds no fonts and emits codes instead of glyphs — detected by a 56-page Chinese prospectus whose output was numeric until recovery and inheritance were added.

4.3 LaTeX to MathML

Answers are Markdown with LaTeX: blocks are recognised first (fences, headings, lists, quotes, tables, display math), then inline spans (inline code, emphasis, links, inline math). The translator covers a defined subset — fractions, roots, superscripts and subscripts, big operators with limits, integrals including contour and surface forms, matrices and `cases`, accents and vector arrows, `\mathbb` and `\mathcal` variants, and `\left…\right` delimiters — emitting MathML token elements for numbers, operators, and alphabetic runs. Recognition accepts `$…$`, `$$…$$`, `\(\u2026\)`, and `\[\u2026\]`, and — the case that motivated the work — **bare LaTeX** with no delimiter, such as a line beginning `\frac{…}{…}` or a block beginning `\begin{aligned}`; before the translator existed such output was displayed verbatim, which users reported as "garbled formulas". Dollar amounts are protected, so `$100` and `$200` are not mistaken for mathematics, and on failure the source is shown unchanged. We chose MathML over bundling KaTeX [9] because the bundle then carries no third-party renderer code, the browser's own math layout is used, and the subset needed for model answers is small enough to test exhaustively. Layout uses a system math font in the order STIX Two Math, Latin Modern Math, Cambria Math, STIXGeneral [13]; the trade-off is coverage, as expressions outside the subset degrade to text.

4.4 Conversation history

History is one JSON document in the page origin's `localStorage` under `dsh.chat-mode.history.v1`, holding the conversation list, each conversation's messages, and its model selection. Caps are 40 conversations, 200 messages per conversation, and 3 500 000 characters; per stored message, 40 000 characters of extracted attachment text and 20 000 of reasoning text are kept, and images keep a 160-pixel thumbnail. Writes are debounced at 600 ms, and when the quota refuses a payload the oldest conversations are dropped until it fits, with the panel reporting the loss. Streaming updates touch only the current message object, so other rows do not re-render. Page-origin storage needs no host storage service and survives restarts on a fixed port, but clearing site data erases it and a `dsh web` instance on another port keeps a separate history; a restored image is a thumbnail, so resending an old turn marks the image unavailable rather than silently sending a degraded picture.

5 Superman Memory

5.1 Storage schema

The memory is a directory `<root>`, by default the harness home's `superman` folder, holding `index.json` and `files/<id>`. The index carries a version marker and item and note arrays. An item holds an id (the first 16 hex digits of the SHA-256 of its bytes), name, category (`pdf`, `image`, `code`, or `other`), media type, byte count, addition time, source (`chat`, `harness`, or `superman`), optional origin, a detail string, text-presence flags, optional image dimensions, and a text preview returned only by `get` and `store`. A note holds an id, kind (`task` or `conversation`), title, body, source, optional session id, timestamps, and up to twelve tags. Bodies are capped at 8 000 characters, previews at 4 000, and 500 notes are retained.

Writes are serialised through a promise chain, each writing `index.json.tmp` and renaming it over `index.json`, so a crash mid-write leaves the previous index intact rather than a truncated one. Duplicates are caught by content hash and name, a repeat upload refreshing the timestamp instead of adding a copy. A damaged index is not fatal: the loader ignores one whose version or shape does not match and rebuilds from files on the next store. The memory thus stays **legible as ordinary files** — because original bytes sit under `files/`, an agent needing a document's full text reads it with its normal file tools.

5.2 Classification cascade

`categorize` runs five stages and stops at the first that decides. **(1) Magic bytes:** `%PDF-` means PDF; PNG, JPEG, GIF, BMP, and RIFF/WEBP signatures mean image. **(2) Declared media type:** `application/pdf` means PDF, `image/*` means image. **(3) Extension:** PDF and image extensions decide those categories, and code extensions — including Mathematica's `nb`, `wl`, and `m` — mean code. **(4) Content heuristics:** a shebang decides code immediately, otherwise up to six patterns (declaration keywords, brace-or-semicolon endings, arrow and range operators, HTML tags, C preprocessor directives, Mathematica delimiters) are tested and a score of at least two means code; before that, control-character density above 10% in the first 4 KiB means other. **(5) Fallback:** remaining files are other.

Image dimensions come from the header without an image library: PNG from the IHDR fields, GIF from the logical screen descriptor, JPEG by walking segment markers to a start-of-frame marker (excluding the Huffman and arithmetic tables), and WebP from the extended-format chunk. The resulting `1920×1080 · 2.4 MB`-style detail appears on the card and in the prompt digest, so a screenshot is identifiable in the agent's prompt without being opened.

5.3 Recall scoring

Recall tokenises the query into lowercase Latin words of two to forty characters plus CJK characters and bigrams, giving Chinese queries surface coverage without a segmenter. An item scores 4 for a token in its name, 1.5 for a token of two or more characters anywhere in name, category, or body (0.6 for a shorter one), 0.5 more for a token occurring more than three times, and up to 1.5 for recency, decaying linearly over thirty days. Notes use a name weight of 3 and a body weight of 1.2 (0.5 for single characters). Results are sorted by score, truncated to a default of six and at most twenty, and each carries a 200-character snippet centred on the earliest match. The response is prompt-ready text, not JSON:

```
■■■■ Superman■■■ N ■■
- [PDF] ■■.pdf■…snippet…
■■■■■■■■■■■■■■■■■■■ files/ ■■■■■■■■
```

This is a lexical design and we treat it as such: deterministic, cheap, and model-free, but limited to surface similarity, so a query that paraphrases the material without sharing tokens returns nothing.

5.4 Prompt injection

Superman registers a `systemPrompt.section` named `superman-memory`, ordered immediately after the file-reference section, whose `text` is a **function**, not a string, so the digest is recomputed at every prompt assembly. It holds the folder path and index location, per-category counts, the twenty-four most recently added items each with an eighty-character snippet, the

six most recently updated records, and an instruction to read files under `files/` when full content is needed; a memory that has not loaded returns an empty string rather than failing assembly. This is why Harness-mode recall needs no explicit call.

5.5 Dual-mode capture

Material reaches the memory by two routes. In chat mode the panel posts bytes and the text it already extracted, so nothing is re-parsed on the host. In Harness mode the host half subscribes to `session/event` and handles three event types: `user/message` records the session's first user text (capped at 300 characters) and scans its content for attachments; `assistant/message` records the latest answer text (capped at 400 characters) and scans for attachments; `turn/end` increments the turn counter and writes a task record with id `session:<sessionId>`, title from the first question, and a body stating the turn count, first question, and most recent answer. Attachment discovery is a bounded recursive walk (depth at most six) collecting objects carrying an attachment id with a media type or name; each is copied once, guarded by a set of ingested ids, images through the attachment service's image reader and other files through its stream reader.

Every step is wrapped so a memory failure cannot break session append: the listener catches, the ingest promise is fire-and-forget, and the note write has an empty rejection handler. That is the right failure direction — a memory that can fail a user's turn is worse than one that occasionally misses a file — but it hides capture failures outside logs. Chat turns also mirror themselves as `conversation` records with id `chat:<conversationId>` and the most recent 8 000 characters of the exchange, which is how one conversation becomes recallable from another.

5.6 Notices

A watcher polls `list` every four seconds and diffs the result against the ids this page has seen. Every unseen item becomes one toast in the `shell.overlay` stack showing the name, category, and detail; it opens the memory panel when clicked and dismisses itself after seven seconds. **The first answer is only a baseline**, and each item is announced at most once, so starting the application does not replay the memory as a burst of notifications. Panel uploads report immediately through the same store; host captures and arXiv archives arrive through the poll.

6 Literature Capture

The arXiv section already ships in DSH; our contribution is a *save to memory* button on each result. It rests on one measured fact: a HEAD request against an `arxiv.org` PDF response returns `access-control-allow-origin: *`, so the page downloads the PDF itself and no host proxy is needed for the bytes — unlike the search feed, which must be proxied. The button fetches in the browser, wraps the bytes as base64, and posts one `store` command with the file name, `application/pdf`, the source `arxiv`, and a text field holding the metadata and abstract, so the paper is both a byte-level artifact and recallable prose.

When the download fails or is blocked the button does not report failure: it posts a `note` command with id `arxiv:<id-or-title>`, the paper title, the metadata-and-abstract body, and the primary category as a tag, so the paper is recallable even though no bytes were stored. Only when the memory itself is unreachable does it report ■■■■ (archiving failed). The visible state sequence is ■■■■■■■ → ■■■… → ■■■ PDF / ■■■■■■■ / ■■■■, so the user can always tell which of the three outcomes occurred. The design prefers a degraded success to a clean failure: storing a record instead of a PDF is a real loss, but the loss is visible in the button label.

7 Evaluation

All numbers here were measured on the working system or read from its source; `papers/data/metrics.json` holds the raw data.

7.1 Source inventory

Figure 2 and Table 2 give the size of the suite: six JavaScript files totalling 7 187 lines and 294 329 bytes. Chat mode dominates at 4 274 lines (4 017 in the client half alone), Superman at 1 988, and the arXiv bundle at 925. The distribution is what the client-half constraint predicts: the browser halves hold 6 121 of the 7 187 lines, because the PDF reader, encoding detector, LaTeX translator, Markdown renderer, memory panel, and search interface all live there.

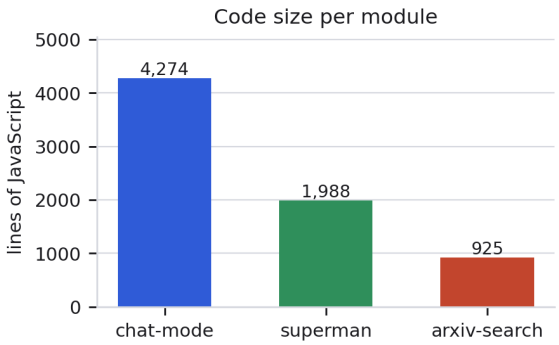


Figure 2. Lines of JavaScript per module.

Table 2. Source inventory of the extension suite.

module	files	lines	bytes
chat-mode	2	4,274	178,186
superman	2	1,988	79,879
arxiv-search	2	925	36,264
total	6	7,187	294,329

7.2 Verification suites

Five offline suites contain 284 checks in 1 696 lines of test code: pipeline 77, formatting and decoding 59, memory host 39, memory panel 57, arXiv 52. Figure 3 and Table 3 show the distribution; all five are green.

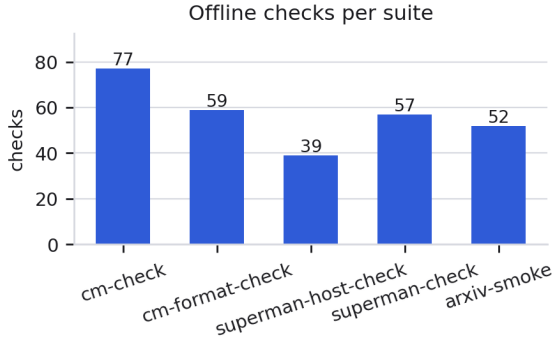


Figure 3. Verification checks per suite.

Table 3. Verification suites, all green.

suite	purpose	checks	lines
chat-mode-check.mjs	pipeline: route, SSE, history, attachments, memory wiring	77	446
chat-mode-format-check.mjs	LaTeX→MathML, Markdown, decoding	59	264
superman-host-check.mjs	classification, store, recall, capture	39	200
superman-check.mjs	panel: entry, folders, preview, upload, notices	57	349
arxiv-smoke.mjs	arXiv bundle incl. bookmark paths	52	451
total		284	1710

The methodology matters more than the count. Every suite drives the shipped files rather than a copy: the browser halves load through a stub module loader standing in for the page's `window.__ModuleLoader__`, with a stub React whose function components are invoked directly, in-memory storage, and a stubbed `fetch`; the host halves run against a fake Cordis context with a temporary memory directory. The suites therefore execute the real parsing, scoring, framing, and command paths. Section 8 states what this cannot prove.

7.3 PDF extraction over a real corpus

Table 4 and Figures 4 and 5 report the corpus run. Fourteen real documents were processed; five yielded text and nine reported `no-text`. The five text-bearing documents produced 199 942 characters, 67 235 of them CJK, across 138 pages. The `noise` column — replacement characters plus C1 controls — is zero for every file, which is what the refusal thresholds of Section 4.2 protect.

Table 4. PDF extraction over the corpus.

file (alias)	KB	ms	chars	CJK	pp	note
brochure.pdf	1507	38.9	9,423	9	19	
document-B.pdf	177	7.9	786	394	2	
prospectus-2025.pdf	576	76.4	90,938	31,791	56	
prospectus-2026.pdf	567	75.1	89,764	31,252	54	
contract-2027.pdf	203	29.0	36,611	0	8	
no text layer	—	—	0	0	0	9 files

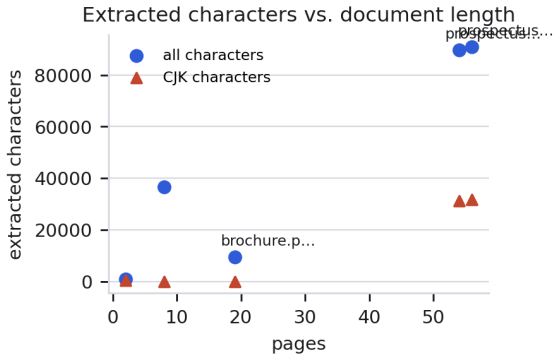


Figure 4. Extracted characters and pages per corpus document.

The two large Chinese documents are 56-page and 54-page graduate admission prospectuses: 90 938 characters (31 791 CJK) and 89 764 (31 252 CJK), both with zero noise. These exercise object streams, inherited resources, and the CJK fallback, and are the two largest extractions by an order of magnitude.

Latency is a median over repeated runs; Figure 5 relates document length to time.

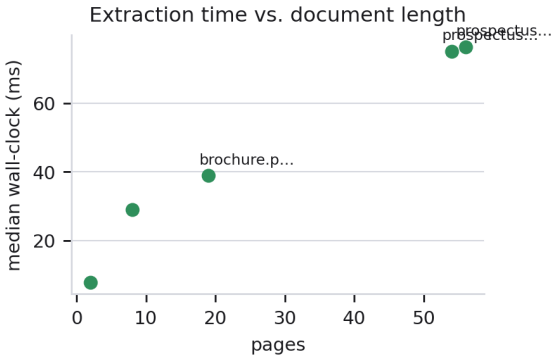


Figure 5. Median extraction time against document pages.

The 19-page document takes 34.7 ms, the 7-page contract 11.7 ms, the 2-page transcript 7.3 ms, and the two 54–56-page Chinese documents 71.8 ms and 71.3 ms; the nine `no-text` files, Apple system PDFs of vector art, return in 0.2–0.8 ms because a page without a text-bearing content stream is refused early. Two observations follow. Cost tracks the volume of text replayed rather than raw file size: the 19-page file is 1.5 MB and takes 34.7 ms, while the 56-page file is 0.59 MB and takes 71.8 ms. And thin text layers are visible: the 19-page file yields 9 423 characters, about 496 per page with nine CJK characters in total, so it is mostly raster imagery.

7.4 Encoding detection

Seven probes cover the cases the detector is built for: UTF-8, UTF-8 with a BOM, GBK bytes, Big5 bytes, UTF-16LE without a BOM, a PNG header, and random high bytes. Table 5 and Figure 6 give the computed plausibility scores; all seven are classified correctly.

Table 5. Encoding probes.

probe	expected	detected	ok
UTF-8	utf-8	utf-8	✓
UTF-8 BOM	utf-8	utf-8	✓
GBK	gbk	gbk	✓
Big5	big5	big5	✓
UTF-16LE (no BOM)	utf-16le	utf-16le	✓
PNG header	binary	binary	✓
random high bytes	binary	binary	✓

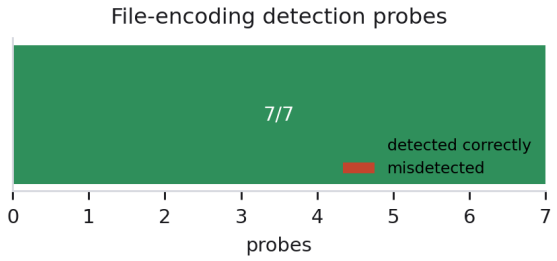


Figure 6. Encoding probes classified correctly.

The two binary probes matter most: the PNG header and the random high-byte file both score 0 under every candidate decoder, fall below the 0.1 acceptance floor, and are reported as binary. Plausible decodings score between 1.0 and 3.0 — Big5 highest at 3.0, then GBK at 2.333, UTF-16LE without a mark at 1.8, UTF-8 at 1.769, and the BOM'd file at 1.0, where the mark itself is not scored. These are outputs of the scoring function, not calibrated thresholds, and their separation from the binary cases (0.0) is the margin in which the 0.1 floor sits.

7.5 Math rendering

Sixteen expressions were translated and validated: fractions, roots, scripts, big operators, integrals including a contour integral, matrices, `cases`, `aligned`, accents, `\mathbb` and `\mathcal`, `\left`…`\right`, and a bracketed identity matrix. All sixteen produce well-formed math markup and none leaves a LaTeX backslash in the output, the failure mode the translator exists to prevent; Table 6 lists the battery with its node counts.

Table 6. Math translation battery.

expression	markup	fractions	scripts	tables	ok
E = mc^2	161	0	1	0	✓
frac{partial f}{partial x	153	1	0	0	✓
vec{F} = mvec{a}	229	0	0	0	✓
sum_{n=1}^{infty} frac{1}	368	2	3	0	✓
int_{-infty}^{infty} e^{-	353	0	3	0	✓
oint_{partial Omega} vec	402	1	4	0	✓
begin{aligned} a &= b + c	281	0	0	1	✓
left[begin{matrix} 1 & 0	315	0	0	1	✓
… 16 total					16/16

The battery is small by design: it enumerates constructs appearing in model answers rather than the full LaTeX math grammar. Markup length ranges from 148 characters for `\mathcal{O}(n \log n)` to 436 for a limit with a telescoping fraction, and structured constructs report the expected fraction and table node counts. What it does not establish is coverage beyond these constructs or visual fidelity of the layout; both are discussed in Section 8.

7.6 Live deployment observations

Beyond the offline suites we inspected the running deployment. The three sidebar entries are active in the served page — `conversation` at order −20, `chat` at −10, `superman` at −5 — alongside the shipped `plugins` entry at order 0, confirming additivity in a live page rather than in a test harness. The overlay stack registers as `shell.overlay[superman.toast]` next to nine shipped occupants of the same slot.

Most directly, the memory's dynamic prompt section was observed inside a running agent's own system prompt, listing the live folder path and the counts `PDF 2 · ■■ 0 · ■■ 0 · ■■ 0 · ■■/■■■■ 2`. Harness-mode recall is thus exercised by the very session in which the work was done: the agent writing this paper could see in its own prompt what the memory held. The two PDF items were a 203.4 KB scanned document and a 5.0 MB arXiv paper [12] stored through the bookmark button.

8 Discussion, Threats to Validity, and Limitations

PDF extraction is a text-layer reader. It does not rasterise or perform optical character recognition, so scanned or outlined documents yield nothing; nine of the fourteen corpus files are that case, and the reader reports `no-text` instead of guessing. The corpus is also small and unsampled: fourteen documents, nine of them vector art from one vendor. The latency figures describe these files on one machine, not a distribution over document kinds.

Recall is lexical. Scoring is token overlap plus recency: deterministic and cheap, and it handles Chinese through characters and bigrams without a segmenter, but limited to surface similarity. A paraphrase with no shared tokens returns nothing, and a query sharing a common token with many items returns a weakly ordered list. No embedding model was measured, so we make no comparison with one.

Chat history is page-origin state. It survives restarts on a fixed origin, but not clearing site data, and it is not shared with a `dsh web` profile on another port. The caps of Section 4.4 also truncate long conversations, silently to anyone not watching the panel's notice. **Restored images are thumbnails**: a 160-pixel thumbnail cannot be sent to a multimodal model as if it were the original, so the panel marks it unavailable when an old turn is resent.

Verification is offline plus live inspection. The suites drive the shipped modules under stubs and the deployment was inspected at the slot, section, and prompt level, but no automated browser-level test operates the DOM. Visual details — spacing, overflow, focus order, and formula rendering in the chosen font — are unverified: a stub React invoking function components directly proves that a component produces an element tree, not that the tree looks right.

The memory grows without a compaction policy beyond caps. Item and note counts are bounded, duplicates are caught by content hash plus name, and previews are truncated, but there is no general deduplication, no age- or relevance-based pruning, and no index compaction; the prompt digest is bounded by construction, so per-turn cost is stable, but the index grows with use.

Measurement scope. All numbers come from one machine and one profile. The 284 checks are assertions in our own suites, not an external conformance suite, and their count should be read as coverage of the paths we thought to test. Two caveats follow from the extension model: because client halves may import only React, the suite reimplements functionality other client packages might have provided, and those reimplementations rest on our own tests; and because host halves need a restart to reload, host paths were exercised in the offline harness more often than in a live restarted process.

9 Conclusion and Future Work

We built and measured a three-part extension suite for an agentic coding harness, and the main result is that the three capabilities a terminal-like harness lacks — a plain conversational surface, durable memory, and literature capture — can be delivered from outside the harness without forking it. The suite occupies four slots additively, registers four exact routes on the harness's authenticated Fetch channel, opens no listening socket, and replaces no shipped interface. It is 7 187 lines of JavaScript across six files, verified by 284 offline checks, and its hand-written PDF reader extracts 199 942 characters from a 14-document corpus with zero noise characters and no third-party libraries.

Four directions follow from the limitations in Section 8.

Embedding-based recall would address the surface-similarity ceiling of the current scorer. The design would keep the lexical path as a deterministic fallback and add a vector index beside `index.json`, so recall degrades rather than breaks when no embedding model is configured; our lexical weights would then serve as a baseline for comparison, which we did not have here.

An OCR fallback would convert the nine `no-text` documents from honest refusals into usable text. The natural shape is a second stage that rasterises pages only when the text layer is empty, is invoked explicitly rather than automatically, and reports per-page confidence so the existing refusal threshold still applies to its output.

Cross-origin memory synchronisation would lift the page-origin storage limitation and let a memory be shared

between the desktop application on one port and a `dsh web` instance on another. Because the memory already lives as ordinary files under a stable directory, the interesting part is conflict handling — two writers appending to one index — not byte transport.

Packaging as a profile bundle collection would make installation one step rather than three. The bundles already share a frozen contract and a common envelope; what is missing is a collection manifest installing them together and expressing the optional dependency between chat mode and the memory, which currently degrades silently when either half is absent.

Acknowledgements

The author thanks the DeepSeek Harness and Cordis projects for the extension framework this work builds on, and the maintainers of the specifications cited below.

References

1. DeepSeek Harness — https://github.com/deepseek-ai/deepseek-harness
2. Cordis plugin framework — https://github.com/cordiverse/cordis
3. ISO 32000-1:2008, *Document management — Portable document format — Part 1: PDF 1.7*.
4. Adobe Systems, *PDF Reference, 6th edition* (FlateDecode, ToUnicode CMaps) — https://opensource.adobe.com/dc-acrobat-sdk-docs/pdfstandards/pdfreference1.6.pdf
5. W3C, *MathML Core* — https://www.w3.org/TR/mathml-core/
6. WHATWG, *Encoding Standard* (GBK, Big5, Shift_JIS, EUC-KR decoders) — https://encoding.spec.whatwg.org/
7. WHATWG, *HTML Living Standard*, § Server-sent events — https://html.spec.whatwg.org/multipage/server-sent-events.html
8. D. E. Knuth, *The TeXbook*, Addison-Wesley, 1984.
9. KaTeX — https://katex.org/
10. S. Robertson and H. Zaragoza, "The probabilistic relevance framework: BM25 and beyond", *Foundations and Trends in Information Retrieval*, 3(4):333–389, 2009.
11. arXiv API user manual — https://info.arxiv.org/help/api/user-manual.html
12. OpenAI et al., "GPT-4 Technical Report", arXiv:2303.08774, 2023. (used only as a corpus sample, not as a design claim)
13. STIX Two Math font — https://github.com/stipub/stixfonts
14. React — https://react.dev/